



ACI™

Generic Simulator for NET24-XPNET

© 2006 by ACI Worldwide Inc. All rights reserved.

All information contained in this documentation is confidential and proprietary to ACI Worldwide Inc., and has been delivered for use pursuant to license. No part of this documentation may be photocopied, electronically transferred, modified, or reproduced in any manner that is contrary to license without the prior written consent of ACI Worldwide Inc.

Access Control Server, ACI Automated Case Management System, ACI Card Management System, ACI Claims Manager, ACI Commerce Gateway, ACI Debit Card System, ACI e-Courier, ACI Host Link, ACI Merchant Accounting System, ACI Payments Manager, ACI Proactive Risk Manager, ACI Smart Chip Manager, BASE24, BASE24-atm, BASE24-card, BASE24-check auth, BASE24-configuration manager, BASE24-es, BASE24-frequent shopper, BASE24-infobase, BASE24-pos, BASE24-refunds, BASE24-remote banking, BASE24-teller, NET24, and WINPAY24 are trademarks or registered trademarks of ACI Worldwide Inc., Transaction Systems Architects, Inc., or their subsidiaries.

Other companies' trademarks, service marks, or registered trademarks and service marks are trademarks, service marks, or registered trademarks and service marks of their respective companies.

Table of Contents

Section 1	Overview	1
1.1	Interface Module	1
1.2	Application Module.....	1
Section 2	Interface Module	3
2.1	pre^process^msg	3
2.2	post^process^msg	3
Section 3	Application Module	5
3.1	Template Files	5
3.1.1	Logical Record	5
3.1.2	Field Description.....	5
3.1.3	Field Types.....	6
3.1.3.1	Programmatically Generated Fields.....	6
3.1.3.2	Date and Time Fields.....	11
3.1.3.3	Constant Fields.....	13
3.1.3.4	BIT MAP Support.....	18
3.2	Transaction Files	19
3.2.1	Logical Record	19
3.2.2	Field Description.....	19
3.3	Command Files.....	21
3.3.1	Standard Processing	21
3.3.2	MATCH Buffer Processing	38
3.3.3	WRITE File Processing	41
3.3.4	Multiple Transaction Processing	43
3.4	Compare Program Support.....	47
3.5	Configuration Examples.....	49
3.5.1	Generic Simulator as Device.....	49
3.5.2	Generic Simulator as BASE24 Authorizer.....	51
3.5.3	Generic Simulator as Device Handler	51
3.5.4	Generic Simulator as Device (Multi-Transaction).....	52
3.5.5	Generic Simulator as Host	53
Section 4	Database Compare Program	55
4.1	Overview	55
4.2	Program Invocation.....	56
4.3	GSCOMP Commands.....	57
4.4	GSCOMP Reports	58
4.4.1	Detail Report	58
4.4.2	Delta Report	59
4.4.3	Regression Error Report	59
4.5	GSCOMP Command File Examples.....	60

ACI Worldwide Inc.

Section 1 Overview

The ACI Generic Simulator is designed to be used in all test phases. It is built on an HP-NSK architecture and runs as a NET24-XPNET satellite process. Events to be simulated are defined by text files. This allows for integration with other simulator applications.

There are two distinct modules to the Generic Simulator architecture, the Interface Module and the Application Module.

1.1 Interface Module

The Interface Module communicates with the HP-NSK message system. The Interface Module is application-insensitive. It can be bound with any Application Module that follows the rules of the specifications. This module does not care about any specific transaction flows, or how the applications or databases are impacted by the transaction.

All communications with the operating system are handled through this module. This includes initialization, inter-process communications, event logging, and all Guardian I/O that the process requires. This module insulates the Application Module from any knowledge about the environment in which it is operating. It also provides hooks for the Application Module.

User exits are provided by the Interface Module to allow customization of message flows beyond the Application Modules design. These user exits are described in further detail in Section 2.

1.2 Application Module

The Application Module was designed as the core of the Generic Simulator capable of generating various message formats and transaction scenarios based on the contents of three files: SIMTPLTS, SIMTRANS, and SIMCMNDS. These files are described in further detail in Section 3.

Additionally, the Application Module performs functionality that includes:

- Handling the message formatting and event sequencing.

The Application Module is environment-insensitive. It can simulate a device, a device handler, an authorization process, or an interface while running as a NET24-XPNET satellite.

- Capturing of before and after images of specified data files.

Images are captured before and after a given transaction scenario (the scenario could be multiple transactions). These images are written to a designated file called the Baseline Transaction File (BTF).

- Resetting of specified records in the BASE24 database from a static, previously defined database. This database is referred to as the Golden Database, or GDB.
- Providing a mechanism for invoking a standalone compare program from within the Generic Simulator SIMCMNDS file. See Section 3.4 for further information regarding the Generic Simulators' support of the compare program.

This compare program, a separate executable named the Generic Simulator Database Comparison Program (GSCOMP), provides a mechanism for comparing the results of test scenarios. This program is designed as a standalone process (it can be invoked from TACL or via Generic Simulator commands). This program is described in more detail in Section 3.

Section 2 Interface Module

Two user exits are provided by the Interface Module to allow customized message handling beyond the scope of the Application Module. To utilize the user exits, create a bindable (code 100) library file using one or both of the user exit procedures as entry points into the custom message handling. Then bind in the new library using HP-NSK's BIND utility, or use the library as a Guardian run time library.

For information on creating a bindable library object file, a Guardian run time library, or using a Guardian run time library, please consult HP-NSK documentation.

The user exits may be used to perform complex tasks such as applying a Message Authentication Code (MAC) to an outbound message.

2.1 pre^process^msg

The pre^process^msg procedure is called with a pointer to the incoming message to allow analysis of and modification to the message prior to processing by the Application Module.

The TAL declaration of the procedure follows:

```
proc pre^process^msg( msg, msg^lgth ) extensible;  
  int      .msg;  
  int      .msg^lgth;
```

- msg is a pointer to the incoming message that includes the NET24-XPNET Network Message Header.
- msg^lgth is a pointer to the length, including the NET24-XPNET Network Message Header, of the incoming message. If the processing alters the overall length of the message, the msg^lgth should be updated with the new length.

2.2 post^process^msg

The post^process^msg procedure is called with a pointer to the outgoing message to allow analysis of, and modification to, the message. This call occurs after processing by the Application Module is complete and before the outbound message is sent to the network.

The TAL declaration of the procedure follows:

```
proc post^process^msg( msg, msg^lgth, txt^lgth ) extensible;  
int      .msg;  
int      .msg^lgth;  
int      .txt^lgth;
```

- msg is a pointer to the current message that includes the NET24-XPNET Network Message Header.
- msg^lgth is a pointer to the length, including the NET24-XPNET Network Message Header, of the incoming message. If the processing alters the overall length of the message, the msg^lgth should be updated with the new length.
- txt^lgth is a pointer to the length, excluding the Network Message Header, of the text of the current message,. The Generic Simulator uses this value to update the Network Message Header in the event that the message size changes within the user exit.

Section 3 Application Module

3.1 Template Files

The template files, which are referenced as SIMTPLTS in the Generic Simulator syntax, define the format of the message. Up to 50 template records can reside in a single file and be active at one time. Template files can be switched in and out by a user or batch commands for greater flexibility.

The SIMTPLTS file is a text file (code 101), which can be created by any text editor. A logical record may consist of one or more field descriptions.

3.1.1 Logical Record

The format of a logical record is:

```
:{<recname>}  
<field description>  
{<field description>}  
:  
{<field description>}
```

The colon (:) is a start of a record indicator.

<recname> is an optional 16-byte identifier used to identify the record in various commands (e.g. SEND, WRITE, MATCH). It must be alphanumeric and start with a letter. If no <recname> is specified the record can only be referenced via its chronological position within the file.

3.1.2 Field Description

The format of a field description is:

```
<name> <format> [<contents>] {comment}
```

<name> is 1-16 alphanumeric characters uniquely identifying the field within the logical record. This name also provides the basis for linking fields from the SIMTRANS file.

<format> is a string specifying how the data is to be formatted.

<contents> specifies the default value; [<value>], minimum, maximum, and increment values [<min>, <max> {, <incr>}], or other special conditions the generated field can have. The maximum length for this field is 195 bytes.

{comment} is an optional comment not containing the special character “}”.

Blank lines are treated as white space.

Lines starting with the right bracket “}” are treated as comments and disregarded.

The maximum length for a line in the template file is 240 bytes.

3.1.3 Field Types

Fields can be grouped into three basic types:

1. Programmatically generated fields (Section 3.1.3.1).
2. Date and Time fields (Section 3.1.3.2).
3. Constant fields. (Section 3.1.3.3).

Bit-mapped fields are a special case identified by names starting with “BIT_”. All field names starting with “BIT_” are reserved (see Section 3.1.3.4).

3.1.3.1 Programmatically Generated Fields

The contents of programmatically generated fields are generated each time they are used in a message that is transmitted (or written to a file) by the Generic Simulator. Global counters and registers are used to store the data used by the Generic Simulator.

The ‘nnn’ in the descriptions that follow represent the length of fixed-length fields. It can be from 1 to 3 numeric bytes representing any positive value less than or equal to 999.

If the format string is not recognizable, the record is invalid. If the record is invalid, it is marked accordingly, and no more transactions will be formatted using the invalid record.

If the transaction fails, no message/record is transmitted.

For programmatically generated fields the basic syntax is:

<name> <format> [<min value>, <max value> {, <incr value>}]

<name> is an alphanumeric field up to 16 bytes long that uniquely identifies the field within the record.

<format> is one of the following (the minimum, maximum, and increment values are dependent upon which format is selected):

NOTE: The second byte (following the %) is case-sensitive.

%BA ⁿⁿⁿ	BLOCK COUNT - FIXED LENGTH ASCII NUMERIC FIELD The value that is the current number of blocks will be converted to ASCII. The result will be placed into a field with length ⁿⁿⁿ .
%B ^{annn}	BLOCK COUNT - FIXED LENGTH EBCDIC NUMERIC FIELD This is the same as previously described but the result is converted to EBCDIC
%BH ⁿⁿⁿ	BLOCK COUNT - FIXED LENGTH PACKED DECIMAL FIELD This is the same as previously described but the result is represented as ⁿⁿⁿ bytes of packed data (Hex)
%BI	BLOCK COUNT - INTEGER NUMERIC FIELD This is the same as previously described but the result is a one-word binary field
%BD	BLOCK COUNT - INT(32) NUMERIC FIELD This is the same as previously described but the result is a two-word binary field
%BF	BLOCK COUNT - FIXED NUMERIC FIELD This is the same as previously described but the result is a four-word binary field
%Ci ^{Ann}	COPY FIXED LENGTH ASCII NUMERIC FIELD The value that is in global counter [i] in the table of counters is converted to ASCII format. If the conversion fails, the record is invalid.
%Ci ^{Hnn}	COPY FIXED LENGTH PACKED DECIMAL FIELD As above, but the final result is represented as ⁿⁿⁿ bytes of packed data.
%Ci ^l	COPY INTEGER NUMERIC FIELD As above, but the final result is a one-word binary field.

%CiD	COPY INT(32) NUMERIC FIELD As above, but the final result is a two-word binary field.
%CiF	COPY FIXED NUMERIC FIELD As above, but the final result is a four-word binary field.
%MAnnn	RANDOM FIXED-LENGTH ASCII W/MOD10 CHECK • <min value> must be ≥ 0 • <max value> must be a value that can be represented as a fixed and $\geq$ <min value>, otherwise the record is invalid. Generate a random number between <min value> and <max value> inclusive. The random number must pass a MOD10 check. Convert the resulting value to ASCII. If a number meeting the requirements cannot be generated, or if the conversion fails, the record is invalid.
%Mannn	RANDOM FIXED-LENGTH EBCDIC W/MOD10 CHECK As above, but the final result is expressed in EBCDIC.
%MHnnn	RANDOM FIXED-LENGTH PACKED DECIMAL W/MOD10 CHECK As above, but the final result is expressed as nnn bytes of packed data.
%QiiAnnn	ACCUM TOTAL - FIXED LENGTH ASCII NUMERIC FIELD The value that is in the global register ii (where ii is 2 numeric bytes in the range 00-39) for accumulating totals will be converted to ASCII. The result will be placed into a field with length nnn.
%Qiiannn	ACCUM TOTAL - FIXED LENGTH EBCDIC NUMERIC FIELD This is the same as previously described but the result is converted to EBCDIC.
%QiiHnnn	ACCUM TOTAL -FIXED LENGTH PACKED DECIMAL FIELD This is the same as previously described but the result is represented as nnn bytes of packed data (Hex.)
%QiiI	ACCUM TOTAL - INTEGER NUMERIC FIELD This is the same as previously described but the result is a one-word binary field.

%QiiD	ACCUM TOTAL - INT(32) NUMERIC FIELD This is the same as previously described but the result is a two-word binary field.
%QiiF	ACCUM TOTAL - FIXED NUMERIC FIELD This is the same as previously described but the result is a four-word binary field.
%RAnnn	RANDOM FIXED LENGTH ASCII NUMERIC FIELD • <min value> must be ≥ 0 • <max value> must be a value that can be represented as a fixed and $\geq$ <min value>. Otherwise the record is invalid. Generate a random number between <min value> and <max value> inclusive. Convert the resulting value to ASCII. If the conversion fails, the record is invalid.
%RHnnn	RANDOM FIXED LENGTH PACKED DECIMAL FIELD As above, but the final result is represented as nnn bytes of packed data.
%RI	RANDOM INTEGER NUMERIC FIELD As above, but the final result is a one-word binary field.
%RD	RANDOM INT(32) NUMERIC FIELD As above, but the final result is a two-word binary field.
%RF	RANDOM FIXED NUMERIC FIELD As above, but the final result is a four-word binary field.

%SiAnnn	SEQUENTIAL FIXED LENGTH ASCII NUMERIC FIELD • <min value> must be ≥ 0 • <max value> must be a value that can be represented as a fixed and $\geq$ <min value>. Otherwise, the record is invalid. If global counter[i] in the table of counters is less than <min value>, set it to <min value>, otherwise increment it by the specified <incr value> (default is 1). If the resulting value is greater than <max value>, set it to <min value>. Convert the resulting value to ASCII.
%Siannn	SEQUENTIAL FIXED LENGTH EBCDIC NUMERIC FIELD As above but the final result is converted to EBCDIC.
%SiHnnn	SEQUENTIAL FIXED LENGTH PACKED DECIMAL FIELD As above, but the final result is represented as nnn bytes of packed data.
%SiI	SEQUENTIAL INTEGER NUMERIC FIELD As above, but the final result is a one-word binary field.
%SiD	SEQUENTIAL INT(32) NUMERIC FIELD As above, but the final result is a two-word binary field.
%SiF	SEQUENTIAL FIXED NUMERIC FIELD As above, but the final result is a four-word binary field.

%XiAnnn	SEQUENTIAL FIXED LENGTH ASCII NUMERIC FIELD WITH ARBITRARY STARTING VALUE. • <min value> must be ≥ 0 • <max value> must be a value that can be represented as a fixed and $\geq$ <min value>. Otherwise, the record is invalid. The initial value of the global counter[j] is set to an arbitrary value $\geq$ the <min value> and $\leq$ the <max value>. If global counter[j] in the table of counters is less than <min value>, set it to <min value>, otherwise increment it by the specified <incr value> (the default is 1). If the resulting value is greater than <max value>, set it to <min value>. Convert the resulting value to ASCII. If the conversion fails, the record is invalid.
%XiHnnn	SEQUENTIAL FIXED LENGTH PACKED DECIMAL FIELD WITH ARBITRARY STARTING VALUE As above, but the final result is represented as nnn bytes of packed data.
%XiI	SEQUENTIAL INTEGER NUMERIC FIELD WITH ARBITRARY STARTING VALUE As above, but the final result is a one-word binary field.
%XiD	SEQUENTIAL INT(32) NUMERIC FIELD WITH ARBITRARY STARTING VALUE As above, but the final result is a two-word binary field.
%XiF	SEQUENTIAL FIXED NUMERIC FIELD WITH ARBITRARY STARTING VALUE As above, but the final result is a four-word binary field.

3.1.3.2 Date and Time Fields

The format for the Date field is:

<name> <format> [<default value>]

<name> is an alphanumeric field up to 16 bytes long that uniquely identifies the field within the record.

<format> is one of the following:

%TAx	SYSTEM DATE-ALPHA ASCII. Fixed length alphanumeric.
%Tax	SYSTEM DATE-ALPHA EBCDIC. Fixed length alphanumeric.
%THx	SYSTEM DATE-PACKED DECIMAL. Fixed length packed decimal.
%TNx	SYSTEM DATE-NUMERIC ASCII. Fixed length numeric.
%Tnx	SYSTEM DATE-NUMERIC EBCDIC. Fixed length numeric.

x is one of the following values:

1 = MMDDYY	9 = MMDDhhmmss
2 = YYMMDD	A = MMDDYYYY
3 = YYDDMM	B = YYYYMMDD
4 = MMDD	C = YYYYDDMM
5 = DDMM	D = YYYYMM
6 = YYMM	E = MMYYYY
7 = MMY	F = DDMMYYYY
8 = DDMMYY	

The <default value> for date fields is one of the following:

- [], [0] message is filled with the current date.
- [+/-<value>] current date plus or minus the number of days specified.
- [<date>] message is filled with the date specified.

The format for the time field is:

<name> <format> [<default value>]

<name> is an alphanumeric field up to 16 bytes long that uniquely identifies the field within the record.

<format> is one of the following:

%J	CURRENT GMT JULIAN TIMESTAMP. A 64-bit Julian Timestamp based on the current GMT.
%tAx	SYSTEM TIME-ALPHA ASCII. Fixed length alphanumeric.
%tax	SYSTEM TIME-ALPHA EBCDIC. Fixed length alphanumeric field.
%tHx	SYSTEM TIME-PACKED DECIMAL. Fixed length packed decimal.

%tNx	SYSTEM TIME-NUMERIC ASCII. Fixed length numeric.
%tnx	SYSTEM TIME-NUMERIC EBCDIC. Fixed length numeric.

x is one of the following values:

- 1 = HHMMSShh
- 2 = HHMMSS
- 3 = HHMM

The <default value> for time fields is one of the following:

- [], [0] message is filled with the current time.
- [<time>] message is filled with the time specified.

3.1.3.3 Constant Fields

The format for constant fields is:

<name> <format> [<default value>]

<name> is an alphanumeric field up to 16 bytes long that uniquely identifies the field within the record.

<default value> is the value that will be associated with the field unless it is overridden by a field in the SIMTRANS file. Everything between the first “[” on the line and the last “]” on the line is treated as data. The maximum length for this field is 195 bytes.

<format> is one of the following:

%Annn	ALPHA ASCII DATA A fixed length alphanumeric field of length nnn. Values will be blank-filled on the right. If the value is too long, the transaction fails.
%annn	ALPHA EBCDIC DATA A fixed length alphanumeric field of length nnn. Values will be blank-filled on the right. If the value is too long, the transaction fails.
%Nnnn	NUMERIC ASCII DATA A fixed length numeric field of length nnn. Values will be zero-filled on the left. If the value is too long, the transaction fails.

%nnnn	NUMERIC EBCDIC DATA A fixed length numeric field of length nnn. Values will be zero-filled on the left. If the value is too long, the transaction fails.
%Vnnn	VARIABLE-LENGTH ASCII DATA Maximum length is nnn. If the value is too long, the transaction fails.
%vnnn	VARIABLE-LENGTH EBCDIC DATA Maximum length is nnn. If the value is too long, the transaction fails.
%Hnnn	FIXED-LENGTH PACKED-DECIMAL FIELD Value will be packed into nnn bytes, zero filled on the left. If the conversion fails or the value is greater than (nnn * 2) bytes long, the transaction fails.
%hnnn	VARIABLE-LENGTH PACKED-DECIMAL FIELD Value will be packed into the minimum number of bytes possible, zero filled on the left. Maximum length is nnn. If the conversion fails or the value is greater than (nnn * 2) bytes long, the transaction fails.
%I	INTEGER FIELD Convert the value to a one-word integer. If the conversion fails, the transaction fails.
%D	INT(32) FIELD Convert the value to a two-word integer. If the conversion fails, the transaction fails.
%F	FIXED FIELD Convert the value to a four-word integer. If the conversion fails, the transaction fails.
%En	ENCRYPTED ASCII FIELD Value must be 16 bytes long or the transaction is invalid. Encrypt it using global key[n]. If there is a problem with the encryption, log a message and send the value in the clear.
%en	ENCRYPTED BINARY FIELD As above, but pack the variable into 8 bytes in the message.

%Gn ENCRYPTED GRAPHIC FIELD
Behaves as the ENCRYPTED ASCII field. However, instead of '0' – '9', 'A' - 'F' (%h30 - %h39, %h41 - %h46), the binary values are %h30 - %h3F. That is '0' – '9', ':', ';', '<', '=', '>', and '?', are transmitted. This is commonly used to transmit hexadecimal values in ASCII in ATM native messages.

Note: In the above formats, 'nnn' represents the length of fixed-length fields and the maximum length of variable-length fields. It can be from 1 to 3 numeric bytes representing any positive value less than or equal to 999.

3.1.3.3.1 Length Attributes

Any of the Constant field formats may have a length attribute placed in front of them. The format of the length attribute is:

#Acnn, #Ecnn, #Fcnn, or #Dcnn

- # identifies this as a length attribute
- A indicates that the lengths are represented in ASCII and represent the number of bytes of data
- E indicates that the lengths are represented in EBCDIC and represent the number of bytes of data
- F indicates that the attribute is the length of the FIELD, i.e., the number of bytes of data in the formatted message
- D indicates that the attribute is the length of the DATA placed in the field, i.e., the length of the ASCII representation of a packed or binary field, or the length of the variable component of a fixed length ASCII or EBCDIC field
- c represents the length of the length attribute
- nn represents the maximum length of the data portion of the field

Each of the length attribute formats are illustrated in the following examples:

Example 1) BINARY LENGTH ATTRIBUTE - DATA LENGTH

TRACK2 #D237%h19 [;1234567890123456=9912?]

- The data is 23 bytes long (which is less than the 37-byte maximum specified in the length qualifier AND will pack into the 19-byte maximum in the format indicator AND can be represented in the 2 bytes specified in the length attribute) so it is OK to continue.

- Since this is a type 'D' length attribute, convert the length to an integer, and put it in the next two bytes of the message.
- Valid values for the data to be packed are %h30 through %h3F ('0' – '9', ':', ';', '<', '=', '>', and '?'). Convert the above special characters to their hexchar equivalents 'A' – 'F', add a leading '0' since the length is odd, and pack the string into the next 12 bytes of the message.

The result (in hex): 00 17 0B 12 34 56 78 90 12 34 56 D9 91 2F

Example 2) BINARY LENGTH ATTRIBUTE - FIELD LENGTH

TRACK2 #F219%h19 [;1234567890123456=9912?]

- The data is 23 bytes long (which will pack into the 19-byte maximum specified in the length qualifier AND will pack into the 19-byte maximum in the format indicator AND can be represented in the 2 bytes specified in the length attribute) so it is OK to continue.
- Since this is a type 'F' length attribute, calculate that it will take 12 bytes of packed data to represent the 23 byte value. Put 12 (represented as an integer) in the next two bytes of the message.
- Valid values for the data to be packed are %h30 through %h3F ('0' – '9', ':', ';', '<', '=', '>', and '?'). Convert the above special characters to their hexchar equivalents 'A' – 'F', add a leading '0' since the length is odd, and pack the string into the next 12 bytes of the message.

The result (in hex): 00 0C 0B 12 34 56 78 90 12 34 56 D9 91 2F

Example 3) EBCDIC LENGTH ATTRIBUTE

TRACK2 #E325%a25 [;1234567890123456=9912?]

- The data is 23 bytes long (which is less than the 25-byte maximum specified in the length qualifier AND less than the 25-byte maximum in the format indicator) so it is OK to continue.
- Since this is a type 'E' length attribute, convert 25 to EBCDIC and put '025' (represented in hex as F0 F2 F5) in the next three bytes of the message.
- Put the value into the next 25 bytes of the message (left justified blank filled on the right).

The result (in hex):

```
F0 F2 F5 5E F1 F2 F3 F4 F5 F6 F7 F8 F9 F0
F1 F2 F3 F4 F5 F6 7E F9 F9 F1 F2 6F 40 40
```

Example 4) ASCII LENGTH ATTRIBUTE

```
TRACK2    #A237%V37      [;1234567890123456=9912?]
```

- The data is 23 bytes long (which is less than the 37-byte maximum specified in the length qualifier AND less than the 37-byte maximum in the format indicator AND “23” can be represented in the 2 bytes specified in the length attribute) so it is OK to continue.
- Since this is a type ‘A’ length attribute, convert 23 to ASCII and put “23” (represented in hex as 32 33) in the next two bytes of the message.
- Put the value into the next 23 bytes of the message.

The result (in hex):

```
32 33 3B 31 32 33 34 35 36 37 38 39
30 31 32 33 34 35 36 3D 39 39 31 32 3F
```

3.1.3.3.2 Check Digit Calculation

This syntax provides for the ability to specify a check digit routine that will be applied to contents of a given field. The format identifier is %K. The %K format identifier is followed by two digits that specify a check digit routine (potentially 10 different routines numbered 0-9) and a global counter (1 of the 10 currently available). Following the two digits identifying which routine and global counter to use will be the actual format of the field (using one of the available formats).

An example field declaration:

```
field_a    %K14A8    [12345678]
```

Based on the above declaration check digit routine 1 (see table for list of currently supported routines) is applied to the alpha ASCII field whose contents is “12345678”. The resultant check digit value is stored in global counter 4.

To place the resultant check digit into another field the existing format identifier for copy (%C) is used. An example of the format:

```
field_b    %C4A1     [ ]
```

Based on this declaration the check digit value is stored in an alpha ASCII field whose length is 1. The content comes from the data stored in global counter 4 (this is where the check digit value for field_a had been stored).

Supported Check Digit Routines

- 0: n/a
- 1: Algorithm based on Section 2.2 Code Values of APPENDIX TWO from the 1996 ACH RULES manual (pg. OR 46)
- 2-9: n/a

3.1.3.4 BIT MAP Support

All field names beginning with "BIT_" (BIT_MAPP, BIT_MAPS, and BIT_1 through BIT_128) are reserved. All of the constant format types can be used in the declaration of these fields. The bitmap fields, BIT_MAPP and BIT_MAPS, must be 64 bits in length.

The reserved word BIT_MAPP represents the primary bit map, BIT_MAPS represents the secondary bitmap, and BIT_1 through BIT_128 represent the corresponding fields in the message.

For outgoing messages the Generic Simulator will verify that the bit for the specified field (named BIT_1 through BIT_128 in the SIMTPLTS record) is turned on in the BIT_MAPP or BIT_MAPS bitmap fields. If the corresponding bit n is set in the bitmap field the BIT_n field is included in the building of the message (the BIT_n field must be declared in the specified record). It is possible that certain BIT_n fields could be defined in the SIMTPLTS record and not be included in the outgoing message unless the bit is set in the bitmap field.

For incoming messages the MATCH NAME (for more information about this, and other commands see Section 3.3.1) functionality is used to identify specified fields. To determine if a match is successful the Generic Simulator moves through the list of fields as defined by the SIMTPLTS record (which is part of the syntax for MATCH NAME <fld name> <template> <text> <cmd buffer id>). The Generic Simulator will verify that the bit for the field is set in the bitmap before checking it against the name in the match table (i.e. the <fld name> parameter). If the bit is on and the name matches the processing continues based on the remainder of the command syntax. All fields that are present in the incoming message must have their corresponding bit turned on in the bitmap fields.

3.2 Transaction Files

The keyword for transaction files is SIMTRANS in the Generic Simulator command syntax. The content of the SIMTRANS file determines the values (if any) that are used to override the default values in the SIMTPLTS file. A single SIMTRANS file may have up to 100 logical records (transactions) in it, each record consisting of up to 40 variables (i.e. fields) of up to 195 bytes each.

Since the SIMTRANS file is an edit file, the entire file is read into memory (i.e. TRAN table) when it is opened, allowing random access to up to 100 transaction records. The content of the TRAN table is accessible by various user commands sent from the NCS or other batch commands.

The maximum length for a line in the transaction file is 240 characters.

Note: A logical record can be made up of multiple field descriptions.

3.2.1 Logical Record

The format of a logical record is:

```
:{<recname>}  
<field description>  
{<field description>}  
:  
{<field description>}
```

The colon (:) is a start of record indicator.

<recname> is an optional 16-byte identifier used to reference the record in SEND, WRITE, or XWRITE commands. It is alphanumeric and starts with a letter. If no name exists the record can only be referenced via its chronological position within the file.

3.2.2 Field Description

The format of a SIMTRANS field description is:

```
<name> [<value>] {comment}
```

<name> is 1-16 alphanumeric characters starting with an alpha character, left justified, blank terminated on the right.

<value> is 1-195 bytes of ASCII characters, delimited on the left by a '[' and on the right by a ']' in the field description. Additional constraints on <value> may apply, depending on the format of the matching field in the template. For example, if the field is going to be formatted into an integer, it should contain numeric data only.

The {comment} is optional and cannot include a ']'.

There is at most one field definition per field description. Blank lines are treated as white space and skipped. Lines where the first non-blank character is a '[' are treated as comments and skipped.

Everything between the first '[' and the last ']' is considered to be part of the variable. The variable may contain '[' or ']'. If you want to put a ']' in your comment, the comment must be on a line by itself, starting with '['.

The record will be marked as invalid for any of the following reasons:

- A logical record contains more than 40 variables.
- A variable is more than 195 bytes long.
- <name> is greater than 16 bytes long.
- <name> is not present.
- There is not at least one matched set of brackets '[' and ']' in the file.
- There is any text between <name> and the first '['.

3.3 Command Files

The keyword for the command file name is SIMCMNDS. The default prefix is '!', where '?' represents:

Space ' ' for comment lines
'A' or 'a' for Generic Simulator commands
'C' or 'c' for Compare Program commands

The first byte of the prefix is configurable by a SET COMMANDPREFIX command to allow for processing messages that start with something other than the default character '!'.

Curly brackets ('{' and '}') in the command syntax delimit parameters in the commands that are optional. The curly brackets are not a part of the actual command. Angle brackets ('<' and '>') surround all parameter names, and are not a part of the actual command.

The maximum length for a line in the command file is 240 characters.

3.3.1 Standard Processing

The following commands are the standard set of commands that are supported in the SIMCMNDS file.

- ACCUM FIELD <id> <rec offset> <rec value> <fld offset> <fld length>

The keyword ACCUM indicates that the following information will be used to configure an accumulated total. The keyword FIELD indicates the type of total (in this case, field totals) that will be tracked. The remainder of the command syntax allows for the configuration of the global register id, the record type, and the specific field within the record that will be used in the accumulation of the total. The configurable parameters are as follows:

<id> identifies the global register that will contain the accumulating value (valid range 0-39)

<rec offset> identifies the position (0-based offset) in the record

<rec value> is the value that will be used (at the offset determined by <rec offset>) to properly identify the record type

<fld offset> identifies the position (0-based offset) of the field that will be used to accumulate a total

<fld length> is the length of the field at the position as determined by the
<fld offset>

- ACCUM HASH <id> <rec offset> <rec value> <fld offset> <fld length>

The keyword ACCUM indicates that the following information will be used to configure an accumulated total. The keyword HASH indicates the type of total (in this case, hash totals) that will be tracked. The remainder of the command syntax allows for the configuration of the global register id, the record type, and the specific field within the record that will be used in the accumulation of the total. The configurable parameters are identical to those used in the ACCUM FIELD command.

- ACCUM REC <id> <rec offset> <rec value>

The keyword ACCUM indicates that the following information will be used to configure an accumulated total. The keyword REC indicates the type of total (in this case, record totals) that will be tracked. The remainder of the command syntax allows for the configuration of the global register id and the record type that will be used in the accumulation of the record total. The configurable parameters are as follows:

<id> identifies the global register that will contain the accumulating value
(valid range 0-39)

<rec offset> identifies the position (0-based offset) in the record

<rec value> is the value that will be used (at the offset determined by <rec offset>) to properly identify the record type that will be counted

- ACCUM RESET <num>

This command allows the user to reset a specific entry (i.e. information used for accumulating a total) in the table (or the entire table). This information for accumulating the total was originally configured using the ACCUM FIELD, ACCUM HASH, or ACCUM REC commands.

<num> identifies which entry in the table to reset (the valid range is 0 to 39). A <num> value of "*" indicates that the entire table is to be reset. When "*" is used the global record count is also reset to zero.

- ASSIGN BTF <filename> {SAVE}

This command causes the Generic Simulator to attempt an open of <filename>. If the <filename> does not exist, it will be created. The BTF is the Baseline Transaction File that will contain the record image information. The optional parameter SAVE can be used if the existing records in the BTF should be left intact. The default functionality is to purge the BTF.

- ASSIGN CAPTURE <filename> {SAVE}

This command causes the Generic Simulator to attempt an open of <filename>. If the <filename> does not exist, it will be created. The optional parameter SAVE can be used if the existing records in the file should be left intact. The default functionality will be to purge the file.

- ASSIGN SIMCMNDS <filename> -- or --
- ASSIGN SIMCMDS <filename>

The Generic Simulator logs a message, opens <filename> and begins processing commands until a message/record is transmitted or a file error occurs. If a file error other than EOF occurs, the Generic Simulator logs an error and closes the file. Otherwise, processing continues until EOF is reached. Upon successful completion the Generic Simulator closes <filename> and logs a message.

Blank lines and lines beginning with the first byte of the command prefix (!) are treated as comments and disregarded.

For each command in the SIMCMNDS file, the command prefix is verified, the command is parsed, and the appropriate actions are taken based on the specified command.

If the ASSIGN SIMCMNDS command is encountered within a SIMCMNDS file (i.e. nesting of SIMCMNDS files) a check is made in order to determine if enough memory exists to process another SIMCMNDS file. If enough memory is not available, the Generic Simulator logs an error (indicating the ASSIGN SIMCMNDS <filename> command for which the problem exists). In addition, the SIMCMNDS file that is currently being processed is abnormally terminated. All commands following the failed ASSIGN SIMCMNDS <filename> command will not be processed.

At startup, the Generic Simulator reads the Logical Network Configuration File (LCONF) for an assign named GENSIM-SIMCMDS. If such an assign exists, the Generic Simulator initializes with this SIMCMDS file as if it were delivered in an NCS 'ASSIGN SIMCMDS' message.

NOTE: When a message is transmitted (via a SEND command), the Generic Simulator suspends processing of the command file until a response is received or a timer expires (see SET DEVICETIMEOUT and SET DEVICEDelay commands). At that time, processing resumes with the next command in the file.

- ASSIGN SIMTPLTS <filename>

Causes the Generic Simulator to open <filename> and build the global TPLT table using the information found in <filename>. This continues until:

- 1) EOF is reached
- 2) Some other file error occurs; the error is logged
- 3) TPLT table is full; an error is logged

If the open was unsuccessful, an error is logged and the file is closed.

See Section 3.1 for more information of the format of the SIMTPLTS file.

- ASSIGN SIMTRANS <filename>

Causes the Generic Simulator to open <filename> and build the global TRAN table using the information found in <filename>. This continues until:

- 1) EOF is reached
- 2) Some other file error occurs; the error is logged
- 3) TRAN table is full; an error is logged

If the open was unsuccessful, an error is logged and the file is closed.

See Section 3.2 for more information on the format of the SIMTRANS file.

- BREAK

Terminates the current REPEAT loop if any. Also 'unlocks' a Generic Simulator process that is 'hung' waiting for a response to a message sent with no timeout.

- CLOSE BTF

The module closes the BTF that is currently open. Each file that was opened by the READFILE command(s) is also closed. Only one BTF can be open by the Generic Simulator at a given time.

- CLOSE CAPTURE

The Generic Simulator closes the CAPTURE file that is currently open. Only one CAPTURE file can be open at a given time.

- COMPTPL <template id> {<variable id>}

This command defines a completion template record and optionally a transaction record.

<template id> is the number or name of the template record to send.

<variable id> is the optional name or number of the variable record that will be used to 'fill in the blanks' in the template record.

The module references the specified record(s) in the global TPLT (and TRAN) table(s).

If the specified records are not present, the command is not valid; a message is logged and no message is transmitted.

For each field in the specified TPLT record the Generic Simulator determines if a corresponding field exists in the associated TRAN record. If it finds a match (based on the field name), it uses the value from the TRAN record; otherwise it uses the default value from the TPLT record. The value is formatted as specified in the field description (see Section 3.1) and placed in the completion message.

- DEFAULT CMD <\$vol.subvol>

This command provides the ability to designate the default volume/subvolume for use when assigning the location of the command files. Once the default volume/subvolume has been specified the syntax for the !A ASSIGN command need only indicate the filename of the applicable file (SIMCMNDS, SIMTPLTS, and SIMTRANS).

- DEFAULT GDB <\$vol.subvol>

This command provides the ability to designate the default volume/subvolume for the Golden Database (GDB) files to reside. See the READFILE command for further information regarding the purpose of the GDB.

- MATCH {NOT} OFFSET <offset> <text> <command buffer>

<offset> is a byte offset into the message.

<text> is a string consisting of quote-delimited text and/or bracket-delimited constants. When brackets are used the format is as follows:

[%f<val>] where:

f is A-ASCII, H-Hex, I-Integer, D-Doubleword, or F-Fixed

<val> is the value to be converted

<command buffer> is a number 0 through 29 which includes identifying the buffer where commands are to be stored.

Note: Brackets may be included in quotes and will be treated as literal brackets, not as delimiters. If literal quotes are to be included, they must be in their hex representation [%H22]. The empty string "" matches everything if it stands alone.

- MATCH PATTERN <string> <text> <command buffer>

<string> and <text> consist of quote-delimited text and/or bracket-delimited constants. When brackets are used the format is as follows:

[%f<val>] where:

f is A-ASCII, H-Hex, I-Integer, D-Doubleword, or F-Fixed
<val> is the value to be converted

<command buffer> is a number, 0 through 29, which identifies the buffer
where commands are to be stored.

Note: Brackets may be included in quotes and will be treated as literal
brackets, not as delimiters. If literal quotes are to be included, they must
be in their hex representation [%H22]. The empty string "" matches
everything if it stands alone. The MATCH NOT logic is not implemented
for this command.

- MATCH NAME <name> <tplt> <text> <command buffer>

<name> is the name of a field in the template file.

<tplt> is the name or number of a template matching the format of the
incoming message.

<text> consists of quote-delimited text and/or bracket-delimited constants.
When brackets are used the format is as follows:

[%f<val>] where:

f is A-ASCII, H-Hex, I-Integer, D-Doubleword, or F-Fixed
<val> is the value to be converted

<command buffer> is a number, 0 through 29, which identifies the buffer
where commands are to be stored. Note that brackets may be
included in quotes and will be treated as literal brackets, not as
delimiters.

Note: If literal quotes are to be included, they must be in their hex
representation [%H22]. The empty string "" matches everything if it
stands alone. The MATCH NOT logic is not implemented for this
command.

- MATCH OFF <command buffer>

<command buffer> is an integer (0 <= <buffer> <= 29) designating a buffer
that is no longer to be tested.

- MATCH END

Any commands that are entered between a MATCH and a MATCH END command are not executed initially but are placed into a command buffer identified by the parameter <command buffer>. A total of thirty buffers, numbered 0 - 29, each containing a maximum of 20 commands will be supported. If a buffer or a record in a buffer overflows, the buffer will be cleared and further commands up to MATCH END or BREAK will be logged and dropped.

Following a successful loading of the commands into the specified command buffers, the Generic Simulator checks each match buffer to determine if a match is found each time it processes an incoming message. If there is a match, the associated commands in the command buffer are processed.

MATCH commands MAY be nested; also, a message may match more than one buffer, and the commands in more than one buffer may be executed during the processing of a single incoming message. Checking of the match buffers (and possible execution of the commands) will begin with the lowest-numbered command buffer.

If a MATCH END is received when no prior MATCH is in effect, a message is logged and the command is dropped.

If the MATCH is nested, MATCH END always applies to the lowest level match.

- READFILE <file> <rel> <image> <key1> <key2> ... <keyn>

This command will provide a mechanism for writing record information from specified files to the Baseline Transaction File (BTF). There are two instances in a transaction scenario where this functionality will typically be invoked:

Prior to initiating the transaction scenario information will be read from the specified files yielding the 'before' record images.

Following the completion of the transaction scenario, information will again be read from the specified files producing the 'after' record images.

The parameters used in this command are as follows:

<file> is the specified file from which the record image will be captured and written to the BTF.

<rel> identifies the particular release of the product for the file whose contents is to be read (e.g. for BASE24 the value would be 50).

<image> describes which image (either the “B”efore or “A”fter) of the specified record is to be stored in the BTF. It also is used to determine if and when the contents of the record is to be reset using the GDB record (accessed via the same key as specified in this command). The file name (i.e. location) for the GDB files is determined using the default volume/subvolume from the DEFAULT GDB command and the <file> parameter from this command.

Note: This includes the CAF, PBF, NEG, UAF files (i.e. the IDF is not used as is the case with the BASE24 file designation.)

The valid combinations of these values are:

- A - store the After image from BASE24 in the BTF
- B - store the Before image from BASE24 in the BTF
- RA - store After image from BASE24 in the BTF; reset the BASE24 record using the contents of the GDB record (with the same key as specified in this command).
- RB - reset the BASE24 record using the contents of the GDB record (with the same key as specified in this command); Store the image from BASE24 in the BTF.
- R - reset the BASE24 record using the contents of the GDB record (with the same key as specified in this command)

<key1> <key2> ... <keyn> parameters are used as the keys to access specific records from the designated file. The READFILE command is currently supported with the following files: ATDD1, CAF, CPF, CUST, IDF, NEG, PBF, PTDD1, PTDF, TDF, TTDF, UAF, SPF, NBF, WHFF. The <key1>, <key2>, ..., <keyn> parameters for the supported files are as follows:

ATDD1

key1 = terminal id

CAF

key1 = fiid (for IDF record)
key2 = cardholder PAN
key3 = cardholder member number

CPF

key1 = prefix
key2 = PAN length
key3 = prefix length

CUST

key1 = customer id
key2 = customer id length

IDF

key1 = fiid

NEG

key1 = fiid (for IDF record)
key2 = cardholder PAN
key3 = cardholder member number

PBF

key1 = fiid (for IDF record)
key2 = account number
key3 = account type

PTDD1

key1 = terminal id

PTDF

key1 = terminal id

TDF

key1 = terminal id

TTDF

key1 = terminal id

UAF

key1 = fiid (for IDF record)
key2 = cardholder PAN
key3 = cardholder member number

SPF

key1 = fiid (for IDF record)

key2 = account number

key3 = account type

key4 = check number (for a range this is the highest check number)

key5 = for a range this is the lowest check number

Note: key5 can be omitted (when a single check number is desired)

NBF

key1 = fiid (for IDF record)

key2 = account number

key3 = account type

key4 = tran date (YYMMDD)

key5 = tran time (HHMMSSSTT)

Notes: 1) key5 can be omitted (the first record for the specified date will be read)
2) both key5 and key4 can be omitted (the first record with the specified account number and type will be read)

WHFF

key1 = fiid (for IDF record)

key2 = account number

key3 = account type

key4 = record type

key5 = amount

key6 = tran date (YYMMDD)

key7 = tran time (HHMMSSSTT)

Notes: 1) key7 can be omitted (the first record for the specified date will be read)
2) both key7 and key6 can be omitted (the first record with the specified amount will be read)

- REPEAT <n>

.

.

REPEAT END

<n> is a valid INT(32) greater than 0 indicating the number of iterations or an asterisk (*) indicating 'do forever' or until a BREAK command is processed.

Any commands that are entered between a REPEAT and a REPEAT END command are not immediately executed but are placed into a command buffer. Up to 20 commands, each up to 240 bytes long, can be supported in the buffer. If the buffer or a record in the buffer overflows, the buffer will be cleared and further commands up to REPEAT END or BREAK will be logged and dropped.

When the Generic Simulator receives a REPEAT END, the commands buffer is processed once and processing then continues with subsequent commands (in the SIMCMNDS file). Thereafter, each time the Generic Simulator processes an incoming message it then executes the commands in the command buffer, until a BREAK command is processed or the specified number of iterations have been processed.

The command buffer is cleared by a REPEAT command, so REPEAT commands may NOT be nested.

- RESEND

The module re-sends the current message to the specified destination.

- SEND [<template id1> {<variable id1>}]
.
.
{ [<template idn> {<variable idn>}] }

The variable id portion is optional within a given set of square brackets (“[” and “]”). If present, a space is used to separate it from the template id.

This command will accept multiple template identifiers (or template/variable pairs) in order to build a record containing variable pieces (e.g. BASE24 Release 5.0 Tokens). Each template (or template/variable pair) must be enclosed in square brackets (“[” and “]”)

<template id1> through <template idn> is the name or number of the template record (from the SIMTPLTS file) to be sent.

<variable id1> through <variable idn> is the name or number of the variable record (from the SIMTRANS file) that will be used to “fill in the blanks” in the template record. If the specified records are not present, the command is not valid; a message is logged and no message is transmitted.

For each field in the specified TPLT record the Generic Simulator determines if a corresponding field exists in the associated TRAN record. If it finds a match (based on the field name), it uses the value from the TRAN record; otherwise it uses the default value from the TPLT record. The value is formatted as specified in the field description (see Section 3.1) and placed in the message.

Execution continues until all entries in the TPLT record have been processed. The message is then sent to the specified destination. The destination is derived from the SET DEST command or an entry in the station table (set via the STA commands).

- SET BLOCKFACTOR <number of records/block>

This command sets the number of records per block value. This value will be used in the calculation of the number of blocks for the %B format (see Section 3.1.3.1) and in determining the number of pad records to write for the XWRITE command (see Section 3.3.3). The default will be to use the file characteristics (record length and block length) to determine this value.

- SET CAPTURE <state>

<state> is one of ON or OFF.

This command sets a flag that will cause the Generic Simulator to write request/response information to a specified file (see ASSIGN CAPTURE).

If the <state> is ON every message that is sent from or received by the Generic Simulator will be written to the specified file, excluding commands.

Setting the <state> to OFF turns off the capture of the request/response messages. The default <state> is OFF; no capture.

- SET COMMANDPREFIX <byte>

<byte> is a single printable ASCII character.

This changes the command prefix for all commands that will be recognized by the Generic Simulator. All commands must start with <byte> to be valid. The default value is the exclamation mark ('!').

- SET CONTROLFLG <state>

<state> is one of ON or OFF.

Setting <state> to ON will cause the simulator to set the MSG^CONTROL^FLAG in the network header to true for outgoing messages. Setting it to OFF will set the MSG^CONTROL^FLAG to false. The default is OFF.

- SET DEST <message destination>

<message destination> is a NET24-XPNET symbolic name. It can be a maximum of 16 bytes long. It will be upshifted by the Generic Simulator.

- SET DEVICETIMEOUT <value>

<value> is a timeout interval in 100'ths of a second. When the Generic Simulator sends a message, a timer is set. It then waits for a response or until the timer expires before proceeding with the processing of the commands. Default is 0; no timer.

- SET DEVICEDelay <value>

<value> is a timeout interval in 100'ths of a second. When the Generic Simulator sends a message, a timer is set; processing does not continue UNTIL the timer expires, regardless of when the response is received. Default is 0; no timer.

Note: DEVICEDelay and DEVICETIMEOUT may not be in effect at the same time. Setting the timeout value for one cancels the other.

- SET DISPLAY <state>

<state> is one of ON or OFF.

ON - Logs all input/output to the logger.

OFF - Does not log input/output.

The default is ON; display all messages.

Additionally, if the <state> is ON the Generic Simulator will validate the contents (i.e. determine if a SIMTPLTS field exists for each field in the SIMTRANS record) of the TRAN table when sending a message. A warning message will be logged for each SIMTRANS variable that is not found in the specified SIMTPLTS record.

- SET FAIL <value>

Where $-1 \leq \text{<value>} \leq 32767$

Sets/resets a message failure flag. All incoming messages, when the <value> is non-zero, will be returned to the sender with MSG^FAILURE in the system header set to <value>. All outgoing messages, when the <value> is non-zero, will set MSG^FAILURE to <value>. A -1 is an allowed value to provide for the simulation of a MSG^FAIL^ACK in the BASE24 environment. Default value is 0; FAIL OFF.

- SET KEY {DOUBLE} <n1> <value> { USE <n2> }

Where $0 \leq \text{<n1>} \leq 9$ and <value> is 16 or 32 Hexadecimal Characters.

DOUBLE is an optional keyword that indicates the key is to be a double length key. A double length key uses 32 Hexadecimal Characters, and the default single length key uses 16 Hexadecimal Characters.

If the optional USE clause is present, $0 \leq \text{<n2>} \leq 9$.

The Generic Simulator has a table of 10 keys used to encrypt variables that are specified as encrypted in the SIMTPLTS file (using the %E, %e or %G formats). This sets key <n1> to <value>. If the optional USE clause is present, it indicates that <value> is encrypted under key <n2>, and is to be decrypted using that key before being stored.

Default is a single length key value of "0123456789ABCDEF".

A double length key uses the left 16 bytes to encrypt, followed by the right 16 bytes to decrypt, followed by the left 16 bytes to encrypt, for each encrypt operation. Similarly, decryption uses the left 16 bytes to decrypt, followed by the right 16 bytes to encrypt, followed by the left 16 bytes to decrypt. The behavior is automated based on the specified 'double' or default 'single' length of the referenced key.

- SET MSGPROWORDS <value>

<value> is one of the following:

- 1) An array of 4 integer values.

If this format is used, all 4 words must be specified.

For example:

'!A SET MSGPROWORDS 18259 0 0 0' sets MSG^PRO^1 to 18259 (i.e. "GS") for ATM device simulation. This command can be used to facilitate the simulation of devices when the Generic Simulator is configured as a process and the device handlers expect the message source to be a station.

'!A SET MSGPROWORDS 1978 0 0 0' sets MSG^PRO^1 to 1978 for POS device emulation and sets the other 3 words to 0.

- 2) Empty; the data is not included in the message.

'SET MSGPROWORDS' disables this processing.

Note: Although the above represents a BASE24 level application, other implementations are possible.

The default is empty.

- SET SEQ <n> <value> {incr}

Where $0 \leq \langle n \rangle \leq 9$ and <value> is a valid INT(32) value ≥ 0 and {incr} is an optional increment value for updating the global counter.

The Generic Simulator has storage for 10 global counters (identified 0-9) where it stores values used in the configuration of programmatically generated fields. This sets global counter <n> to <value> and the increment to {incr}. The default for {incr} is 1.

- SET TOTAL <num> <value>

This command allows the user to set one of the accumulated totals to a specific value. An example of this would be to set a batch total to zero (0) at the beginning of every batch in the file.

<num> identifies which total will be set (valid range 0-39)

<value> is a valid INT(32) value greater than or equal to zero (0)

- SET USERDATA <value>

This command allows the user to configure data that will be included in the user header area of the network message (MSG^UDATA). The <value> can be a maximum of 50 bytes and may consist of bracket-delimited constants or of quote-delimited strings. When brackets are used the format is as follows:

[%f<val>] where:

f is A-ASCII, H-Hex, I-Integer, D-Doubleword, or F-Fixed
<val> is the value to be converted

- SET VARIABLE <tran> <name> [<value>]

<tran> is the name or number (0 <= <tran> <= 99) of the global TRAN table record.

<name> is the name of a variable in the TRAN record specified.

<value> is an ASCII string following the syntax defined for variable values enclosed in [] and will allow negative values.

- STATUS

Displays the current global field values based on the following commands:

```
SET DEST <message destination>
SET COMMANDPREFIX <byte>
SET DISPLAY <state>
SET FAIL <value>
SET MSGPROWORDS <value>
SET DEVICDELAY <value>
SET DEVICETIMEOUT <value>
```

```
Example: INFO - Destination - P1A^SPDH
         INFO - COMMANDPREFIX = !, DISPLAY = -1, FAIL = 0
         INFO - Msgprowords - 1978 0 0 0
         INFO - DEVICETIMEOUT - 3000 milliseconds
```

- The location to where the log messages generated by the Generic Simulator will be switched <log>.

The following commands may be used only in a MATCH command buffer. If they are encountered when commands are NOT being placed in a match command buffer, a message is logged and the command is disregarded.

Allowable forms of '@' are:

@	-	The string for the MATCH in whose buffer this command is being entered
@n	-	The string in match buffer 'n'
@+n, @-n	-	'n' bytes into (or before) the string for the current buffer
@n+m, @n-m	-	'm' bytes into (or before) the string for match buffer 'n'

- DELAY <value>

This command should immediately follow the `FORMAT` command. `DELAY` causes a timer to be set. When the timer expires, the message, as originally built in the `FORMAT` command, is sent.

It may NOT be used with RESPOND.

- The variable id portion is optional within a given set of square brackets (“[” and “]”). If present, a space is used to separate it from the template id.

This command will accept an optional length and multiple template identifiers (or template/variable pairs) in order to build a buffer containing variable pieces (i.e., BASE24 Tokens). Each template (or template/variable pair) must be enclosed in square brackets (“[” and “]”).

<lgth> is the number of bytes from the received message to copy to the beginning of the reply. An asterisk (“*”) is a valid value and is used to indicate that the length of the current (i.e. received) message will be used.

<template id1> through <template idn> is the name or number of the template record to send.

<variable id1> through <variable idn>, if present, is the number or name of the variable record that will be used to ‘fill in the blanks’ in the template record.

This command is typically used with the DELAY command to format the message to be sent after the timer expires.

The Generic Simulator references the specified record(s) in the global TPLT and TRAN tables. If the specified records are not present, the command is invalid; a message is logged and no message is transmitted.

For each field in the specified TPLT record the Generic Simulator determines if a corresponding field exists in the associated TRAN record. If it finds a match based on the field name, it uses the value from the TRAN record; otherwise it uses the default value from the TPLT record. The value is formatted as specified in the field description (see Section 3.1) and placed in the message.

Execution continues until all entries in the TPLT record have been processed. The message is then sent to the source identified in the network header of the incoming message taking into account any specified delay (see DELAY command for more information.)

- MODIFY <offset> <value>

<offset> is an offset in the current message buffer.

<value> is the value to put there. <value> may consist of bracket-delimited constants or of quote-delimited strings. When brackets are used the format is as follows:

[%f<val>] where:

f is A-ASCII, H-Hex, I-Integer, D-Doubleword, or F-Fixed
<val> is the value to be converted

This command is typically used with the RESPOND command

- RESPOND

Sets a flag that causes the Generic Simulator to respond to the source of the current message with the current message buffer. The current message buffer may be modified via one or more MODIFY statements prior to the response.

- SET KEY {DOUBLE} <n1> @ {USE <n2>}

Where $0 \leq \text{<n1>} \leq 9$. If the optional USE clause is present, $0 \leq \text{<n2>} \leq 9$. The Generic Simulator has a table of 10 keys used to encrypt fields that are specified as encrypted in the SIMTPLTS file. This sets key <n1> to the 16 or 32 bytes indicated by @.

If the optional keyword DOUBLE is specified, the specified key is to be a double length key and will reference 32 bytes instead of the default of 16 bytes.

If the optional USE clause is present, it indicates that <value> is encrypted under key <n2>, and is to be decrypted using that key before being stored.

A double length key uses the left 16 bytes to encrypt, followed by the right 16 bytes to decrypt, followed by the left 16 bytes to encrypt, for each encrypt operation. Similarly, decryption uses the left 16 bytes to decrypt, followed by the right 16 bytes to encrypt, followed by the left 16 bytes to decrypt. The behavior is automated based on the specified 'double' or default 'single' length of the referenced key.

- SET SEQ <n> @ {<len>}

Where $0 \leq \text{<n>} \leq 9$.

<value> is a valid INT(32) value ≥ 0 .

The Generic Simulator has 10 global counters, 0-9, where the values are stored that are used by programmatically generated fields. This sets register <n> to the value at @ for <len> bytes. If <len> is not specified, the length in the match buffer is used for OFFSET and PATTERN matches, and the length of the bit field for BITMAP matches.

- SET VARIABLE <tran> <name> @ {<len>}

<tran> is the name or number (0 <= <tran> <= 99) of the global TRAN table record.

<name> is the name of a variable in the TRAN record specified.

<len>, if present, represents the number of bytes to assign. If <len> is not present, the value specified in the MATCH record is used.

3.3.3 WRITE File Processing

The following commands support the writing of records to a specified file. The contents of the record(s) are configured in the SIMTPLTS and SIMTRANS files.

- ASSIGN FILE <filename>

The Generic Simulator opens <filename>. This is the file that will contain records that have been written using the WRITE or XWRITE commands. <filename> must already be created. If the open is unsuccessful, an error is logged.

- CLOSE <filename>

Causes the <filename> to be closed. <filename> must match the filename specified in the ASSIGN FILE command.

- WRITE [<template id1> {<variable id1>}]
 .
 .
 {[<template idn> {<variable idn>}]} }

The variable id portion is optional within a given set of square brackets (“[” and “]”). If present, a space is used to separate it from the template id.

This command will accept multiple template identifiers (or template/variable pairs) in order to build a record containing variable pieces of a given record. Each template (or template/variable pair) must be enclosed in square brackets (“[” and “]”).

<template id1> through <template idn> is the name or number of the template record to be output.

<variable id1> through <variable idn> is the name or number of the variable record that will be used to “fill in the blanks” in the template record.

The Generic Simulator references the specified records in the global TPLT (and TRAN) tables. If the specified records are not present, the command is not valid; a message is logged and no record is written to the output file.

For each field in the specified TPLT record the Generic Simulator determines if a corresponding field exists in the associated TRAN record. If it finds a match (based on the field name), it uses the value from the TRAN record, otherwise it uses the default value from the TPLT record. The value is formatted as specified in the field description (see Section 3.1) and placed in the record to be written to the specified output file.

Execution continues until all entries in the TPLT record have been processed. The formatted record is then written to the file specified in the ASSIGN FILE command.

- XWRITE [<tplt id> {<tran id>}] <num>

This command provides the support for specifying multiple records to be written to a file. The command syntax is as follows:

<tplt id> and the optional <tran id> are supported as they currently are in existing commands within the Generic Simulator (i.e. the WRITE, SEND, and FORMAT commands.)

<num> is used to indicate the number of records (using <tplt id> and <tran id> as the record layout) that will be written to the specified file.

Any sequentially generated fields (including date and time fields) are incremented for each record that is written. The XWRITE command will write records to the file specified via the ASSIGN FILE (same as the WRITE command.)

NOTE: A special case of the XWRITE command: Padding of records within a block can be handled by configuring a pad record in the template file. Once this record is created in the SIMTPLTS file it can be referenced in the XWRITE command. A "P" (or "p") will be used for the value of the <num> parameter. The Generic Simulator will determine the number of records to write (based on the file characteristics) and, using the specified <tplt id> and <tran id> information, write the record(s) to the file.

3.3.4 Multiple Transaction Processing

The following commands provide support for multiple transaction functionality. The Generic Simulator is capable of transmitting large volumes of messages so caution must be used when configuring for these scenarios.

The basis for the following commands is to generate a range of evenly accessed random PAN card numbers and optionally terminal ids in the outgoing message. Each message is sent to a defined (via STA commands), non-busy station in a round-robin fashion.

- SET ACCT1 DELTA <value>

This command provides the ability to specify a value that will be added to (or subtracted from, if negative) the PAN that has been previously generated (via SET PAN PREFIX, SET CARD HI, SET CARD LO, SET CARD INCR commands).

Note: The user is required to insure that the resultant account number is in the proper range for purposes of balancing the system resources. <value> must be numeric.

- SET ACCT1 OFFSET <value>

This command specifies a byte offset (i.e. a location from a base of 0) into the message buffer for placement of the Account 1 information. <value> must be numeric.

- SET ACCT2 DELTA <value>
- SET ACCT2 OFFSET <value>

These commands provide identical functionality as ACCT1 but for a second account number.

Note: Both the Account 1 and Account 2 information is based off the programmatically generated PAN data. If the PAN is not to be included in the message the user must omit the SET PAN OFFSET command (i.e. if the offset has not been specified, no data will be included in the outgoing message buffer). This allows the PAN to be generated, and used to create account information, without requiring that it be included in the message.

- SET CARD LO <value>

This command sets the low end range of the card numbers to be generated. <value> must be numeric and fit into a FIXED field.

- SET CARD HI <value>

This command sets the high end range of the card numbers to be generated. <value> must be numeric and fit into a FIXED field.

- SET CARD INCR <value>

This command sets the increment value of the card number range. <value> must be numeric and fit into an FIXED field.

- SET COMP <ON/OFF>

Tells the Generic Simulator to send completion message after receiving a response. The completion message will be sent to the source of the response. This field is optional and the default is OFF. The command must be sent first if using the COMPTPL template and transaction command.

- SET PAN LEN <value>

This command indicates the length of the PAN within the track 2 of the outgoing message. <value> must be numeric between 1 - 20.

- SET PAN OFFSET <value>

This command indicates the byte offset of the PAN in the outgoing message. <value> must be numeric.

- SET PREFIX <value>

This command sets the prefix of the PAN card numbers that get generated. <value> must be numeric and between 1 to 11 bytes in length.

- SET TERM PREFIX <value>

This command sets a terminal id field in an outgoing message with the given prefix value. <value> may be alphabetic or ASCII numeric from 1 to 12 bytes in length. This parameter is optional and has a default value of 00. In generating terminal ids a numeric suffix, 6 bytes in length, is used to increment the terminal id numbers. So, to generate a terminal id that is 12 bytes in length, a prefix 6 bytes in length would need to be declared.

- SET TERM LO <value>

This command sets the low end range of the terminal ids to be generated. <value> must be numeric and fit into an INT(32).

- SET TERM HI <value>

This command sets the high end range of the terminal ids to be generated. <value> must be numeric and fit into an INT(32).

- SET TERM OFFSET <value>

This command indicates the byte offset of the terminal id field in the outgoing message. <value> must be numeric.

- STA <dest>

This command defines the name of a station id or process name that will serve as the destination for outgoing messages.

<dest> is typically the symbolic name of a station declared in the NEFs.

A maximum of 750 of these destinations can be defined. Messages are sent to these destinations in a round-robin fashion. Busy destinations are skipped until they become available. Busy destinations are set to 'not busy' when the Generic Simulator receives responses from them.

- STA END

This command indicates the end of the station declarations. This command is required after using the STA command.

If simulating a device, configure the terminal related parameters before the card and PAN parameters. This allows for running the Generic Simulator without entering the terminal parameters (i.e. simulating a host or switch). If they are the last parameters defined, the extended memory segment will not be created with the terminal information and user defined DUMPs are likely to happen during running of transactions.

Lastly, the station ids must be defined after the card and optional terminal parameters are set or the STA command will be invalid. The building of the internal tables is based on how many cards and terminals have been configured.

3.4 Compare Program Support

The !C commands that are supported by GSCOMP can also be included in the SIMCMNDS file. (See Section 3 for a complete list of the commands supported by GSCOMP.) All of the !C commands will be ignored by the Generic Simulator (i.e. the parameters are not checked for validity) except for !C COMPARE. When this command is encountered the GSCOMP executable will be invoked. The GSCOMP executable must reside on the same \$vol.subvol as the Generic Simulator object. The Generic Simulator will pass the name of the SIMCMNDS file as input to GSCOMP (GSCOMP produces as part of the output the contents of the entire command file that is being processed). The GSCOMP executable will then process the !C commands in the file to completion as usual. No other communication will take place between Generic Simulator and GSCOMP.

It is possible to invoke multiple GSCOMP processes from within the same SIMCMNDS file (e.g. this could be necessary if more than one BTF has been used to capture data in a two part transaction). To achieve this an additional digit can be added to the command prefix designation (e.g. !C2 ...). It is necessary that each GSCOMP command in a selected group contain the same command prefix. An example is as follows:

The following commands would be placed in the SIMCMNDS file after the transaction scenario has completed and the BTF's (bfile1 and bfile2 in the example) contain valid before and after record images .

```
!C DICT $dvol.dsubvol
!C BTF $bvol.bsubvol.bfile1
!C RECORD CAF
!C REPORT $s.#rpt1
!C COMPARE DELTA
```

[The first occurrence of a group does not need the additional digit in the prefix. The default is group 1.]

```
!C3 DICT $dvol.dsubvol
!C3 BTF $bvol.bsubvol.bfile2
!C3 RECORD PBF
!C3 REPORT $s.#rpt2
!C3 COMPARE DELTA
```

[This group will cause a second GSCOMP process to be started. The GSCOMP process will be notified (via the Generic Simulator) to use group 3 commands. All of the commands designated with a 3 will be used to create the output from GSCOMP.]

The valid range of groups is 1 through 9. This results in a maximum of 9 separate GSCOMP processes that can be invoked from a single command file.

3.5 Configuration Examples

3.5.1 Generic Simulator as Device

- A. Define request message (SPDHRQST in SIMTPLT1.)
- B. Define two SIMTRANS records (TRAN1 and TRAN2 in SIMTRAN1.)
- C. Build SIMCMNDS file to send two messages (varying amount and track2 fields). Set the destination to be P1A^SPDH (the device handler.) Set the timeout to 30 seconds.

- A. Define the request message

SIMTPLT1:

```

]      Begin logical record 1
      :SPDHRQST
]      Device type
      DEV_TYPE  %A2          [9.]
]      Transmission number, sequential, counter 1, length 2
      XMIT_NUM  %S1A2        [1,99]
]      Terminal id, 16 bytes, left justify, blank fill
      TERM_ID   %A16         [90000000]
]      Clerk id, 6 bytes, blank fill
      CLERK_ID  %A6          [ ]
]      Date/time, 12 bytes, blank fill
      DAT_TIME  %A12         [ ]
]      Financial type
      TYPE      %A1          [F]
]      Online subtype
      SUBTYPE   %A1          [O]
]      Purchase trancode
      TRANCODE  %N2          [00]
]      Processing flags
      PROCFLGS  %N3          [000]
]      Response code, let it default to zeroes
      RESPCODE  %N3          [ ]
]      Amount, variable length, 18 bytes max, default $10.00
      AMT_FS    %H1          [1C]
      AMT_FID   %A1          [F]
      AMOUNT    %V18         [1000]
]      Track2, variable-length ASCII, 40 bytes max
      TRK_FS    %H1          [1C]
      TRK_FID   %A1          [q]
      TRACK2    %V40         [;1234567890123456=9912?]

```

B. Define SIMTRANS records.

SIMTRAN1:

```
] Begin logical record 1
:TRAN1
  TRANCODE [00]
  AMOUNT   [9900]
  TRACK2   [;9876543210987654=9912?]
] Begin logical record 2
:TRAN2
  TRANCODE [00]
  AMOUNT   [1234]
  TRACK2   [;1010101010101010=9912?]
```

C. Build SIMCMNDS file.

SIMCMNDS:

```
!A ASSIGN SIMTPLTS $<vol>.<subvol>.SIMTPLT1
!A ASSIGN SIMTRANS $<vol>.<subvol>.SIMTRAN1
!A SET DEST P1A^SPDH
! Timeout = 30 seconds
!
!A SET DEVICETIMEOUT 3000
! Display requests and responses
!
!A SET DISPLAY ON
! Send the two transactions in SIMTPLT1 and SIMTRAN1
!
!A SEND [SPDHRQST TRAN1]
!A SEND [SPDHRQST TRAN2]
```

3.5.2 Generic Simulator as BASE24 Authorizer

SIMCMNDS file to configure the Generic Simulator as POS 3.4 AUTH.

```
!A SET DISPLAY ON
!  
! Is it a request? Put commands in command buffer 0
!  
!A MATCH OFFSET 0 "0200" 0
!  
! Change message type
!  
!A MODIFY 0 "0210"
!  
! Set response code
!  
!A MODIFY 349 "000"
!  
! Generate approval code
!  
!A MODIFY 451 "APPCDE B"
!A RESPOND
!A MATCH END
```

No SIMTPLTS or SIMTRANS records are needed in this scenario. The Generic Simulator responds with the same (including the configured modifications) buffer to the source of the incoming message.

3.5.3 Generic Simulator as Device Handler

SIMCMNDS file to configure the Generic Simulator as an SPDH device handler, handling communication key changes and out-of-sequence responses. The commands to generate the transaction are not included. Note the nested matches and negation.

```
!A SET KEY 0 <ptdf master key>
:
:
! If response code is 'out of sequence', set flag to
! override, and resend
!A MATCH OFFSET 45 "899" 1
!A SET VARIABLE TRAN1 OVERRIDE [1]
!A SEND [SPDHRQST TRAN1]
!A SET VARIABLE TRAN1 OVERRIDE [0]
!A MATCH END
! If trancode is download, GS<g> is KPE
!A MATCH OFFSET 40 "90" 2
! Look for pattern in incoming message buffer
!A MATCH PATTERN [%H1D]"g" "" 3
! Get the key out of the response and
! decrypt it using the master key
!A SET KEY 1 @+2 USE 0
!A MATCH END
!A MATCH END
! If trancode is not download, FS<M> is KPE
!A MATCH NOT OFFSET 40 "90" 4
!A MATCH PATTERN [%H1C]"M" "" 5
!A SET KEY 1 @+2 USE 0
!A MATCH END
!A MATCH END
```

3.5.4 Generic Simulator as Device (Multi-Transaction)

SIMCMNDS file to configure the Generic Simulator as a device sending messages to a SPDH device handler in multiple transaction scenario.

```
!A ASSIGN SIMTPLTS $<vol>.<subvol>.SIMTPLT1
!A SET DISPLAY OFF
!A SET DEST s1a^spdh
! 1/2 sec delay - 2 tps
!A SET DEVICEDelay 50
! send trans until operator types 'BREAK' at NCS
!A REPEAT *
! all fields are defined in the SIMTPLTS; no SIMTRANS needed
!A SEND [SPDHRQST]
!A REPEAT END
```


3.5.5 Generic Simulator as Host

SIMCMNDS file to configure the Generic Simulator as a HISO Host simulator authorizing transactions.

```
!A ASSIGN SIMTPLTS $<vol>.<subvol>.SIMTPLT2
!A ASSIGN SIMTRANS $<vol>.<subvol>.SIMTRAN2
! Determine if this is a request. HISORQST is the template record that
! defines the incoming message. Put commands in match buffer 0
!
!A MATCH NAME MSG_TYP HISORQST "0200" 0
! Save the audit trace number
!
!A MATCH NAME BIT_11 HISORQST "" 1
!A SET VARIABLE TRANRESP AUDITNUM @ 6
!A MATCH END
! Save track2; default to length of bit field
!
!A MATCH NAME BIT_16 HISORQST "" 2
!A SET VARIABLE TRANRESP TRACK2 @-2
!A MATCH END
! Save the retrieval reference number
!
!A MATCH NAME BIT_37 HISORQST "" 3
!A SET VARIABLE TRANRESP RETRVNUM @ 6
!A MATCH END
! Build the response - HISORESP and TRANRESP are records in
! the SIMTPLT2 and SIMTRAN2 files; delay 2 seconds before sending
!
!A FORMAT [HISORESP TRANRESP]
!A DELAY 200
!A MATCH END
```

ACI Worldwide Inc.

Section 4 Database Compare Program

4.1 Overview

The Generic Simulator Database Compare Program (GSCOMP) is responsible for processing database records stored in the Baseline Transaction File (BTF) and generating reports based on the data (this includes the Regression Baseline Transaction File (RBTF), which is identical in format to the BTF. The Generic Simulator is responsible for the creation and building up of both the BTF and RBTF.

The GSCOMP is designed as a stand-alone program so that it may be invoked from TACL, or from the Generic Simulator via NEWPROCESS.

Information describing the format of the records being compared is retrieved from the Data Dictionary. This allows the GSCOMP program to not be tied directly to a particular release of a database.

The following reports are generated by GSCOMP:

- Detail Report
- Delta Report
- Regression Error Report

These reports are discussed in detail in Section 4.4.

4.2 Program Invocation

The GSCOMP program can be invoked either from TACL or from the Generic Simulator via the SIMCMNDS file.

When the program is invoked from TACL the following syntax is used:

```
TACL> run gscomp/in <cmd-file>/<cmd-prcs-grp>
```

Where:

<cmd-file> = the name of the command file

<cmd-prcs-grp> = the command processing group

When the program is invoked from Generic Simulator, the startup message sent from Generic Simulator will contain the name of the GSCOMP command file in the input file area and the command-processing group as the first character in the parameter area of the startup message. See Section 3.4 for more information.

4.3 GSCOMP Commands

The GSCOMP command file contains the commands that are necessary to properly process the BTF or the BTF and RBTF files. There can be more than one group of commands in a command file. This allows more than one compare session to be run from the same command file. Each group of commands comprises a Command Processing Group.

The commands in the command file are identified by “!C” in the first two characters of a command line. The next character following the “!C” identifies the Command Processing Group to which the commands belong. The valid values for a Command Processing Group is a space and “1” through “9”. If there is a space following the “!C” it is equivalent to “1”. Any line read from the command file that does not contain “!C” in the first two characters is considered a comment. The following commands are supported by GSCOMP:

- COMPARE <type>

This command identifies the type of comparison to be performed.

<type> can be one of DETAIL, DELTA or REGRESS. See Section 4.4 for a description of each of these report types.

- DICT <\$vol.subvol>

This command identifies the volume and subvolume where the data dictionaries for the database are located.

- BTF <\$vol.subvol.filename>

This command identifies the volume, subvolume and filename of the Baseline Transaction File to be used in the compare process.

- RBTF <\$vol.subvol.filename>

This command identifies the volume, subvolume and filename of the Regression Baseline Transaction File to be used in the compare process.

- RECORD <rec-1>, <rec-2>, ..., <rec-n>

This command identifies the records (CAF, PBF, TDF, etc) in the BTF or the BTF and RBTF that are to be compared. There is currently a limit of 20 records.

- REPORT <rpt-loc>

This command identifies the location to where the report is to be sent.

- EXCLUDE <rec>.<fld>

This command identifies the fields within a specified record that are to be excluded from the report even if they are different. The EXCLUDE command is only valid when the type of comparison is DELTA or REGRESS. It is ignored for a comparison type of DETAIL.

If you want to exclude all of the fields in a group you may specify an asterisk (*) after the group to indicate all fields.

For example:

```
!C  EXCLUDE  CAF.SEG0.LAST-FM.*
```

will exclude the printing of all of the groups and fields subordinate to LAST-FM.

The current maximum number of EXCLUDE commands per record is 20. All EXCLUDE commands must follow the RECORD command.

NOTE: When specifying a field to be excluded, make sure that you use a dash (-) instead of a circumflex (^) as a data name connector. For example:

```
caf.seg0.crd-stat NOT caf.seg0.crd^stat
```

4.4 GSCOMP Reports

The GSCOMP module will generate three types of reports. The user will specify which reports to generate via the COMPARE <type> command. The location of the report is determined by the REPORT <rpt-loc> command.

4.4.1 Detail Report

This report is a field-by-field expansion of a specified file record. The report contains “before” and “after” field level data values for each detail field in the specified file. The report is generated from the “before” and “after” record images in a single BTF. Every field of the specified file record is listed and those that have changed are marked accordingly (i.e. with the “<== DIFFERENCE” syntax).

The following is an example layout of the Detail Report:

```
RECORD: <rcd-name>
KEY: <key information>
STRUCT.LVL1.FLD1_1
BEFORE: xxxxxxxxxxxx
AFTER: xxxxxxxxxxxx
STRUCT.LVL1.FLD1_2 <== DIFFERENCE
BEFORE: xxxxxxxxxxxx
AFTER: xxxxxxxxxxxx
```

4.4.2 Delta Report

This report contains “before” and “after” field level data values for each detail field in the specified file that are different. The report is generated from the “before” and “after” record images in a single BTF.

The following is an example layout of the Delta Report:

```
RECORD: <rcd-name>
KEY: <key information>
STRUCT.LVL1.FLD1_1
BEFORE: xxxxxxxxxxxx
AFTER: xxxxxxxxxxxx
STRUCT.LVL1.FLD1_2
BEFORE: xxxxxxxxxxxx
AFTER: xxxxxxxxxxxx
```

4.4.3 Regression Error Report

This report contains the differences between the “after” field level data values for each detail field in the specified file. The report is generated from the “after” record images in a BTF and a RBTF.

The following is an example layout of the Regression Error Report:

```
RECORD: <rcd-name>
KEY: <key information>
STRUCT.LVL1.FLD1_1
BTF: xxxxxxxxxxxx
RBTF: xxxxxxxxxxxx
STRUCT.LVL1.FLD1_2
BTF: xxxxxxxxxxxx
RBTF: xxxxxxxxxxxx
```

4.5 GSCOMP Command File Examples

Detail Report

```
!C   COMPARE  DETAIL
!C   DICT      $DVOL.DSUBVOL
!C   BTF       $BVOL.BSUBVOL.BFILE
!C   RECORD   CAF, PBF, TDF
```

The above commands will generate a detail report of the “before” and “after” CAF, PBF and TDF record images in the BTF file \$BVOL.BSUBVOL.BFILE using the data dictionary on \$DVOL.DSUBVOL.

Delta Report

```
!C   COMPARE  DELTA
!C   DICT      $DVOL.DSUBVOL
!C   BTF       $BVOL.BSUBVOL.BFILE
!C   RECORD   BEG, UAF
```

The above commands will generate a delta report of the “before” and “after” NEG and UAF record images in the BTF file \$BVOL.BSUBVOL.BFILE using the data dictionary on \$DVOL.DSUBVOL.

Regression Report

```
!C   COMPARE  REGRESS
!C   DICT      $DVOL.DSUBVOL
!C   BTF       $BVOL.BSUBVOL.BFILE
!C   RBTF      $RVOL.RSUBVOL.RFILE
!C   RECORD   CAF, PBF, TDF
```

The above commands will generate a regression report of the “after” CAF, PBF and TDF record images in the BTF file \$BVOL.BSUBVOL.BFILE and the RBTF file \$RVOL.RSUBVOL.RFILE using the data dictionary on \$DVOL.DSUBVOL.

Delta Report with Excluded Fields

```
!C   COMPARE   DELTA
!C   DICT      $DVOL.DSUBVOL
!C   BTF       $BVOL.BSUBVOL.BFILE
!C   RECORD    CAF, PBF
!C   EXCLUDE   CAF.SEG0.WDL-PRD
!C   EXCLUDE   CAF.SEG0.LAST-FM.*
```

The above commands will generate a delta report of the “before” and “after” CAF and PBF record images in the BTF file \$BVOL.BSUBVOL.BFILE using the data dictionary on \$DVOL.DSUBVOL.

The delta report will not contain any occurrences of SEG0.WDL-PRD, or any groups or fields that are subordinate to SEG0.LAST-FM in the CAF record.

ACI Worldwide Inc.